Functions in Python

Introduction to Function

- function is a named sequence of statement(s) that performs a computation.
- It contains line of code(s) that are executed sequentially from top to bottom by Python interpreter.
- They are the most important building blocks for any software in Python.

Functions can be categorized as belonging to

- Modules
- Built in
- User Defined

Module

- module is a file containing Python definitions (i.e. functions) and statements.
- Standard library of Python is extended as module(s) to a programmer.
- Definitions from the module can be used within the code of a program.
- To use these modules in the program, a programmer needs to import the module.
- There are many ways to import a module in your program, the one‟s which you should know are:
  - import
  - from

Import Statement

- It is simplest and most common way to use modules in our code. Its syntax is:

  import modulename1 [,modulename2, ---------]

- Example

```
import math
value= math.sqrt (25)
```

From Statement

- It is used to get a specific function in the code instead of the complete module file.
- If we know beforehand which function(s), we will be needing, then we may use from.
- For modules having large no. of functions, it is recommended to use from instead of import.
- Syntax

  from modulename import functionname [, functionname…..]
- Example

```
from math import sqrt
value = sqrt (25)
```

Math Module

- ceil(x) : It returns the smallest integer not less than x, where x is a numeric expression.
- floor(x) : It returns the largest integer not greater than x, where x is a numeric expression.
- fabs(x) : It returns the absolute value of x, where x is a numeric value.
- exp(x) : It returns exponential of x: e^x, where x is a numeric expression.
- log(x) : It returns natural logarithm of x, for x > 0, where x is a numeric expression.

Math Module Function

- log10(x) : It returns base-10 logarithm of x for x > 0, where x is a numeric expression.
- pow(x, y) : It returns the value of xy, where x and y are numeric expressions.
- sqrt (x) : It returns the square root of x for x > 0, where x is a numeric expression.
- cos (x) : It returns the cosine of x in radians, where x is a numeric expression.

Math Module Function

- sin (x) : It returns the sine of x, in radians, where x must be a numeric value.

- tan (x) : It returns the tangent of x in radians, where x must be a numeric value.

- degrees (x) : It converts angle x from radians to degrees, where x must be a numeric value.

- radians(x) : It converts angle x from degrees to radians, where x must be a numeric value.

Random Module

- random ( ): It returns a random float x, such that 0 ≤ x<1
- randint (a, b) : It returns a int x between a & b such that a ≤ x ≤ b
- uniform (a,b): It returns a floating point number x, such that a <= x < b.
- randrange([start,] stop [,step]): It returns a random item from the given range.

Built in Function

- Built in functions are the function(s) that are built into Python and can be accessed by a programmer.

- These are always available and for using them, we don't have to import any module (file).

- abs (x) : It returns distance between x and zero, where x is a numeric expression.

- max(x, y, z, ...) : It returns the largest of its arguments: where x, y and z are numeric variable/expression.

Built-in Function

- min(x, y, z, ...) : It returns the smallest of its arguments; where x, y, and z are numeric variable/expression.

- cmp(x, y) : It returns the sign of the difference of two numbers: -1 if x < y, 0 if x == y, or 1 if x > y, where x and y are numeric variable/expression.

- divmod (x,y) : Returns both quotient and remainder by division through a tuple, when x is divided by y; where x & y are variable/expression.

- round(x [, n]) : It returns float x rounded to n digits from the decimal point, where x and n are numeric expressions. If n is not provided then x is rounded to 0 decimal digits.

Built-in Function

- Some other Bulit-in Function which already discussed:

input()	evel()	len()	int()
float()	ord()	chr()	print()

User Defined Functions

- In Python, it is also possible for programmer to write their own function(s).
- These functions can then be combined to form module which can be used in other programs by importing them.
- Syntax:

```
def <function name>(<parameter list>):
        Statement 1
        ..........
        [return expression]
```

```
def SI(P,R,T):
    return(P*R*T)
Output:
>>> SI(1000,2,10)
20000
```

Parameters and Arguments

- **Parameters** are the value(s) provided in the parenthesis when we write function header. These are the values required by function to work.
-

  Example: def SI (P,R,T):

- **Arguments** are the value(s) provided in function call/invoke statement. List of arguments should be supplied in same way as parameters are listed.

- Example: Arguments in function call

  >>> SI (1000,2,10)

Types of Parameter or Formal Argument

- You can call function by using the following types of formal arguments:
  - Required Argument
  - Keyword (Named) Argument
  - Default Argument
  - Variable-length arguments

Required Argument

- Required argument are the arguments passed to a function in correct positional order.
- Hear number of arguments in the function call should match exactly with the function definition.

Keyword Arguments

- Keyword arguments are the named arguments with assigned values being passed in the function call statement, i.e., when you use keyword arguments in a function call, the caller identifies the arguments by the parameter name.

Default Argument

- Default arguments are those that take a default value if no argument value is passed during the function call.
- You can assign this value by with the assignment operator =.

Variable-length Arguments

- Variable length argument is used when you are not sure how many arguments are going to be passed to a function, or if you want to pass a stored list or tuple of arguments to a function.
- An asterisk (*) is placed before the variable name that will hold the values of all non-keyword variable arguments.
- You can also use the **kwargs when you don’t know how many keyword arguments will be passed to a function.

Python Code to demonstrate *args

```
def find_sum(*values):
    ans=0
    for a in values:
        ans=ans+a
    print("Sum of all NO is :",ans)

find_sum(5,6,7,8,1) # ans is 27
```

Python code demonstrate **kwargs

```
def find_sum(**values):
    for a in values:
        print(a,"  ",values[a])

find_sum(num1=5,num2=5,num3=9)
```

```
================ RESTART: C:/User
num1     5
num2     5
num3     9
```

Anonymous Function

- You can use the lambda keyword to create small anonymous function.
- These functions are called anonymous because they are not declared in the standard manner by using the def keyword.
- An anonymous cannot be a direct call to print because lambda requires an expressions.
- Syntax:
  - func_name= lambda arg1,arg2,…. : expression
- Example:

```
>>> si=lambda amt,rate,time:(amt*rate*time)/100
>>> si(3000,4.5,6)
810.0
>>>
```

The return statement

- A python function could also optionally return a value.
- This value could be a result produced from your function's execution or even be an expression or value that you specify after the keyword 'return'.

```
def square(num):
    return num*num
ans=square(5)
print("Result : ",ans)
```

```
Result :  25
>>>
```

Void Functions

- A function that doesn't return a value is called void function or non-fruitful function whereas a function that return values is called fruitful function.
- A void function internally returns an empty value none.

```
def printTable(num):
    for a in range(1,11):
        print(a,'x',num,'=',(a*num))

printTable(7)
```

Global and Local variables

- Global variable are the one that are defined and declared outside a function and you can use them anywhere.
- Local variables are the one that are defined and declared inside a function/block and you can use them only within that function or block.
- A global variable exists for the life of the program, but local variable created during a function call and are discarded when the function's execution has completed.

```
a=4 # global a
def myFun():
    a=3  # local a
    print(a) # 3

myFun()
print(a) # 4
```

Global Statement

- Global statement are remotely like declaration statement in python.
- They are not type or size declaration, through they are namespace declarations.
- The global statement tells python that a function plans to change one or more global names, i.e. names that live in the enclosing module's scope (namespace).

- *A namespace is a practical approach to define the scope, and it helps to avoid name conflicts.*

Library Functions

- Common Function
  - input()
  - eval()
  - print()
- String Function
  - count()
  - find()
  - rfind()
  - capitalize()
  - title()
  - lower()
  - upper()
  - swapcase()
  - islower()
  - isupper()
  - istitle()
  - replace()
  - strip()
  - lstrip()
  - rstrip()
  - split()
  - partition()
  - join()
  - isspace()
  - isalpha()
  - isdigit()
  - isalnum()
  - startswith()
  - endswith()
  - split()

- Numeric Function
  - Eval()
  - Max()
  - Min()
  - Pow()
  - Round()
  - Int()